RLHF

1. Preliminaries

For the basic structure of the Transformer model, see Transformer Model.
The conventional RLHF consists of two phases: the reward model training and the policy model training.
There are three models involved: The base model, the reward model, and the policy model. The parameters of the base/reference model are fixed throughout the RLHF process. The base model is prompted with prompts x to produce pairs of answers $(y_1, y_2) \sim \pi_{\text{ref}}(y|x)$ for human annotators to express preferences of one over the other, denoted as $y^+ \succ y^-|x$. The reward model and the policy model are usually initialized as the base model in practice.

How to compute the log-likelihood?

We use the log of conditional probabilities throughout the LLM training, and the quantities are the backbone of computing any learning objective.

- Per-token log-probability: after aligning the output digits of a LM over one batch with the next-token targets, computing the cross-entropy, we get the per-token log-prob: $\log p(x_t|x_{<t})$.
- How is $\pi_\theta(y|x)$ actually computed?
  - From the per-token log-prob, apply a response mask, add up the unmasked per-token log-probs, and use the Bayes Theorem to compute the log of the joint probability.

Reward modeling phase:

The first phase in the conventional RLHF, is to model the human preferences based on the generated dataset $\{x_{(i)}, y^+_{(i)}, y^-_{(i)}\}_{i=1}^N$. Assume there is a latent reward model to be learnt: $r(x, y)$, and that the human preference can be modeled as a Bradley-Terry model:

$$p(y_1 \succ y_2|x) = \frac{\exp(r(x, y_1))}{\exp(r(x, y_1)) + \exp(r(x, y_2))}$$

Parameterize the reward model with a neural network $r_\phi(x, y)$, the training loss objective is framed as a negative log likelihood:

$$\mathcal{L}_{\text{reward}}(r_\phi) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}[\log p(y^+ \succ y^-|x)]$$

In practice, the reward model is often initialized from the fixed base model π_{ref} with a linear layer on top of the final transformer layer that produces a single scalar prediction for the reward

value. Prior works also normalize the learnt reward values to ensure a lower variance of the reward function.

RL Phase:

If treating the LM generation process as a game played by the LM, acting as an agent interacting with the environment, the goal is for the LM to learn a policy of replying to prompts. The game state represents a user prompt and all previously generated tokens, and the game action is generating a new token. The simple formalism is an one-step deterministic MDP. Thus, the RLHF is essentially different from the classic reinforcement learning(a multi-step MDP). But the discussion over their differences is not the focus here.
The policy model is parameterized as π_θ, and in practice it is often initialized from the base model. In prior works, the RL optimization is formulated as (with the learnt reward model r_ϕ fixed):

$$\max_{\pi_\theta} \left(\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}[r_\phi(x,y)] - \beta\mathbb{D}_{\mathrm{KL}}[\pi_\theta(y|x)||\pi_{\mathrm{ref}}(y|x)]\right)$$

It aims at finding a policy that maximizes the reward, while avoiding the learnt policy from deviating from the fixed base model too far.
There are several reasons of why imposing the KL-divergence term in the RL optimization. First, if the policy model deviates from the base model too far, the reward computed based on the generated response from the policy model is no longer an accurate reflection of the human preference(the reward model is learnt based on the preference pairs generated from the base model). Second, the KL term helps to maintain the generation diversity and to prevent mode-collapse into single high-reward answers.
Due to the discrete nature of language generation, the objective is not differentiable, and is typically optimized with the reinforcement learning.
The standard approach has been to construct the reward function (essentially a rewriting of the above optimization function)$r(x,y) = r_\phi(x,y) - \beta(\log \pi_\theta(y|x) - \log \pi_{\mathrm{ref}}(y|x))$ and maximize using PPO: $\max_{\pi_\theta}\mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(x,y)]$.

2. Experiment Setups

- Base LM: `Qwen/Qwen2.5-1.5B-Instruct`
- Offline dataset of preference pairs(generated from the base LM, annotated by humans? or GPT-5.4 maybe), also used for reward-model training and testing:
  - `train_prefs` : 4744 pairs of responses, with preference labels;
  - `test_prefs` : 256 held-out pairs of responses, with preference labels.
- Online dataset:
  - `train_gen` : 4744 prompt-only split for online RLHF rollouts, the prompt distributions for `train_prefs` and `train_gen` are the same;
  - `test_prefs` : 256 held-out prompt-only split for policy evaluation.

3. Evaluation Protocol

Primary metric:

- the win rate against the base model under an external LLM judge.
  Material: Public evaluation prompt file & reward-model preference evaluation pairs.

Secondary diagnostics:

Offline:

- held-out preference accuracy on `test_prefs`,
- approximate KL of the current policy to the frozen base model,
- generation-collapse diagnostics and qualitative sample quality

Online:

- reward-model pair accuracy on `test_prefs`,
- reward-model score improvements.

4. Algorithm Part

Notation

- x is a prompt;
- y is a full response sequence;
- y^+ and y^- denote the preferred and rejected responses;
- π_θ is the current policy/LM model;
- π_{ref} is the frozen reference policy/base model;
- $\sigma(\cdot)$ is the logistic function;
- $\beta > 0$ is the preference-loss scale parameter.
  Define the reference-corrected preference margin:

$$\triangle_\theta(x, y^+, y^-) = \log(\frac{\pi_\theta(y^+|x)}{\pi_\theta(y^-|x)}) - \log(\frac{\pi_{\text{ref}}(y^+|x)}{\pi_{\text{ref}}(y^-|x)})$$

  The preference margin quantifies by how much amount the current policy bias more towards the preferred response to rejected response than the fixed reference policy.

Offline Methods

1. DPO:

The key contribution of the algorithm is to transform a loss function over reward functions into a loss function over policies. The change-of-variable approach avoids fitting an explicit & standalone reward model and optimizes directly over policies, bypassing the RL step completely. The policy objective(minimization) is:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}[\log \sigma(\beta \triangle_\theta(x, y^+, y^-))]$$

2. IPO:

Implicit Preference Optimization uses the same reference-corrected margin $\triangle_\theta(x, y^+, y^-)$ as DPO, but instead of applying a logistic loss, it matches the margin to a target value:

$$\mathcal{L}_{\text{IPO}}(\pi_\theta; \pi_{\text{ref}}) = \mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}\left(\triangle_\theta(x, y^+, y^-) - \tfrac{1}{2\beta}\right)^2$$

The intuition for the method is: DPO keeps pushing to separate chosen and rejected responses through a logistic objective; while IPO treats the desired preference margin as a regression target. This often makes IPO feel more conservative than DPO: rather than endlessly increasing margins, it aims for a particular scale.
IPO is therefore a useful comparison point because it changes the shape of the loss while keeping almost the same ingredients as DPO.

3. AOT:

Alignment via Optimal Transport(AOT) changes the granularity of comparison. Instead of only optimizing each chosen/rejected pair independently, it compares the distribution of chosen scores against the distribution of rejected scores across a minibatch.
Each example (corresponding to a prompt x) produces a scalar reference-corrected reward:

$$r_\theta(x, y) = \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

The formula is precise up to a normalization constant with respect to x.
AOT sorts the chosen rewards $r_\theta(x, y^+)$ and the rejected rewards $r_\theta(x, y^-)$ separately and then applies a DPO-style penalty to the sorted quantile gaps:

$$\mathcal{L}_{\text{AOT}} = \mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}}\left(-\log \sigma(\beta(r^+_{(i)} - r^-_{(i)}))\right)$$

Here $r^+_{(i)}$ is the i-th sorted chosen reward and $r^-_{(i)}$ is the i-th sorted rejected reward. Intuitively, AOT encourages the entire chosen distribution to shift upward relative to the rejected distribution, not only each individual paired example.

Reward Modeling:

The Bradley-Terry reward model.

Online Methods

0. KL regularization:

Online methods regularized the trainable policy toward the frozen reference policy. At one token position, define

$$\triangle(y_t) = \log\pi_{\text{ref}}(y_t|x, y_{<t}) - \log\pi_\theta(y_t|x, y_{<t}).$$

The KL divergence is $\mathbb{D}_{KL}[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)] = \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}[-\Sigma_{t=1}^{T}\triangle(y_t)]$. So if using the sampled-token nonnegative KL proxy:

$$\hat{k}(a) \doteq \exp\triangle(a) - \triangle(a) - 1$$

The KL divergence proxy can be written as:

$$\hat{\mathbb{D}}_{KL}[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)] = \Sigma_{t=1}^{T}\hat{k}(a)$$

Notes:

1. The expectation of the KL proxy under $y \sim \pi_\theta(\cdot|x)$ is the exact token-level KL:

$$\mathbb{D}_{KL}[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)] = \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}[\Sigma_{t=1}^{T}\hat{k}(a)]$$

2. By Taylor expansion, easy to verify that KL proxy is non-negative.

1.1 GRPO:

The algorithm comes from the paper DeepSeekMath, in which they built a small-but-strong LLM model for math reasoning with meticulous data curation for pre-training and a new RLHF algorithm GRPO.

GRPO is derived from an actor-critic RL algorithm PPO(Policy Gradient Optimization). For details about PPO, see here: PPO. But in this note, we only compare the differences of GRPO and PPO in terms of the model structure.

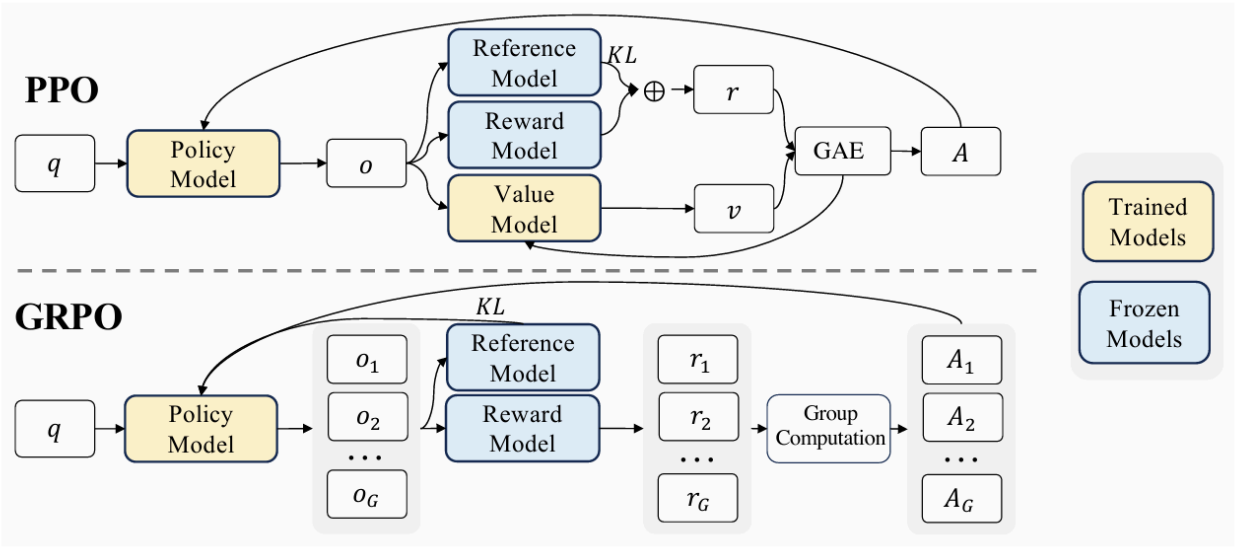

Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

PPO:

The optimization objective for PPO is to maximize the smaller term in the unclipped and clipped terms:

$$\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_{\theta_{\text{old}}}(y|x)}\{\frac{1}{T}\Sigma_{t=1}^{T}\min[\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})}A_t,\text{clip}(\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})},1-\epsilon,1+\epsilon)A_t]\}$$

In practice, there's also a KL penalty term:

$$\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_{\theta_{\text{old}}}(y|x)}\left(\frac{1}{T}\Sigma_{t=1}^{T}\min[\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})}A_t,\text{clip}(\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})},1-\epsilon,1+\epsilon)A_t]\right)-\beta\mathbb{D}_{KL}(\pi_\theta||\pi_{\text{ref}})$$

Notes:

1. The fraction $\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})}$ and the expectation over the old policy model $\pi_{\theta_{\text{old}}}$ is derived from the importance sampling of the off-policy learning(the vanilla PPO is on-policy, which means in every step of the gradient ascent, we need to sample new prompt-answer pairs from the current policy model).
2. If ignoring the KL term and the clipped gradient term, take the derivative of the objective function with respect to θ, we recover a formula very similar to the policy gradient of the off-policy PPO:

$$\nabla_\theta J(\theta)=\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_{\theta_{\text{old}}}(y|x)}\left(\frac{1}{T}\Sigma_{t=1}^{T}\left(\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x,y_{<t})}\cdot\nabla_\theta\log\pi_\theta(y_t|x,y_{<t})\cdot A_t\right)\right)$$

3. A_t is the advantage, and is computed by applying Generalized Advantage Estimation(GAE). GAE requires training a Value Model alongside the Policy Model.
4. Using a clipped surrogate objective is to stabilize the training process(an engineering trick) by preventing the policy from changing too much within one updating step. Note that the clipped gradient surrogate is different from the gradient norm clipping trick commonly used

to prevent exploding gradients.

GRPO:

The GRPO bypasses the Value Model, and uses a Group-relative advantage in place of GAE. For one prompt x_i, suppose we sample G responses with rewards $r_{i,1}, \cdots, r_{i,G}$. The group-relative advantages:

$$
\begin{aligned}
A_{i,j} &= \frac{r_{i,j} - \mu_i}{\sigma_i + \epsilon} \\
\text{with } \mu_i &= \frac{1}{G} \Sigma_{j=1}^{G} r_{i,j}, \\
\sigma_i &= \sqrt{\frac{1}{G} (r_{i,j} - \mu_i)^2}.
\end{aligned}
$$

1.2 DrGRPO:

DrGRPO makes two changes relative to GRPO:

1. it uses mean-centered group-relative advantages without dividing by the per-group standard deviation;
2. it removes the per-sequence length normalization inside the surrogate.
   The motivation is that these changes alter how GRPO handles sequence length and reward scaling. In particular, removing the per-sequence normalization changes the length dependence of the update, which makes DrGRPO a useful comparison point when you are thinking about length bias in policy optimization.

1.3 GSPO:

Instead of clipping the token-level likelihood ratios as GRPO does, GSPO aggregates token log-ratios into a length-normalized single sequence-level ratio, and then applies the PPO-style clipping at the sequence level. This changes the optimization geometry: GSPO treats the response as one structured action rather than a bag of token-wise policy updates.